

Mattia Robuschi Caprara

Structured Query Language



Prefazione

!!ATTENZIONE!!

Questo documento è un semplice OCR delle slide del prof.

Contents

1	Introduzione	3
2	Definizione dei dati	3
3	Domini	3
3.1	Character	3
3.2	Bit	3
3.2.1	Intervalli temporali	3
3.3	Tipi numerici esatti	4
3.4	Tipi numerici approssimati	4
3.5	Data e Ora	4
4	Definizione di Domini	4
4.1	Esempio:	4
5	Definizione di Schemi	5
6	Definizione di Tabelle	5
6.1	Esempio	5
7	Valori di default	5
8	Vincoli intrarelazionali	5
8.1	Esempio	6
8.2	Vincoli di Integrità Referenziale	6
8.3	Esempio MySQL	6
8.4	Esempio MariaDB	6
9	Modifica degli schemi	7
9.1	Comando alter	7
9.2	Comando drop	7
10	Inserimento	7
10.1	Esempio	7
11	Cancellazione	8
12	Modifica	8
12.1	Esempio	8
13	Interrogazioni	9
13.1	Esempio	9
13.2	Interpretazione algebrica delle interrogazioni	9
13.3	Interrogazioni su più tabelle	10
14	Clausola where	10
15	Operatore like	10
16	Gestione dei valori nulli	11
17	Join Implicito ed Esplicito	11
18	Duplicati	11

19 Join: sintassi	12
20 Alias e Variabili	12
20.1 Esempio di alias usato come variabile	12
21 Ordinamento	12
22 Operatori Aggregati	13
23 Interrogazioni con raggruppamento	13
23.1 group by	14
24 Operatori aggregati e Raggruppamenti	14
25 Raggruppamenti e Target List	14
26 Condizioni sui gruppi	15
26.1 Esempio	15
26.2 Where o Having?	15
27 Select	15
28 Interrogazioni di tipo insiemistico	15
28.1 Esempi	16
29 Interrogazioni nidificate	16
29.1 Interrogazioni nidificate per gli alias	17
29.2 Interrogazioni nidificate e join	17
29.3 Operatori Aggregati e Nidificazione	17
29.4 Interpretazione delle query nidificate	18
30 exists	19
31 Costruttore di tuple	19
32 Vincoli di integrità generici	20
32.1 Esempio check	20
33 Asserzioni	20
34 Viste	21
35 Controllo degli accessi	22

1 Introduzione

SQL (pronunciato anche come l'inglese sequel): acronimo di Structured Query Language (linguaggio di interrogazione strutturato)

Linguaggio completo che presenta anche proprietà di:

- DDL (Data Definition Language)
- DML (Data Manipulation Language)

Con SQL è quindi possibile:

- Definire schemi di basi di dati
- Eseguire query
- Modificare il contenuto della base di dati

2 Definizione dei dati

Esistono 6 domini elementari:

- Character
- Bit
- Tipi numerici esatti
- Tipi numerici approssimati
- Data e ora
- Intervalli temporali

3 Domini

3.1 Character

Rappresenta singoli caratteri o stringhe

```
1 character [varying] -- [(Lunghezza)]
2 character set NomeFamigliaCaratteri
3
4 character(20) -- stringa di lunghezza 20
5 character varying(100) -- stringa di lunghezza max. 100
6 character(20) character set Greek -- stringa di lunghezza 20 in caratteri greci
7
```

3.2 Bit

rappresenta attributi o vettori di attributi che possono assumere solo valori 0 o 1.
Utilizzato per implementare flag.

```
1 bit [varying] [(Lunghezza)]
2
```

3.2.1 Intervalli temporali

```
1 interval UnitaDiTempo1[(Precisione)] [to UnitaDiTempo2[(Precisione)]]
2
```

UnitaDiTempo1 e *UnitaDiTempo2* specificano le unità di misura da usare dalla più lunga alla meno lunga
NB Definizione valida nei 2 sotto-intervalli [*year, month*] e [*day, second*].

```
1 interval day(4) to second(6) -- rappresenta l'intervallo [0,10000) giorni e secondi con
2 precisione al milionesimo di s.
```

3.3 Tipi numerici esatti

Valori interi o con parte decimale di lunghezza prefissata

```
1 numeric [(Precisione [,Scala])]
2 decimal [(Precisione [,Scala])]
3 integer
4 smallint
5
```

Precisione specifica il numero di cifre significative

Scala quante cifre si rappresentano dopo la virgola

```
1 decimal(4) -- numeri da -9999 a 9999
2 numeric(6,3) -- numeri da -999.999 a 999.999
3
```

La precisione degli interi dipende dall'implementazione

3.4 Tipi numerici approssimati

rappresentano valori reali approssimati

```
1 float [(Precisione)]
2 double precision
3 real
4
```

Precisione specifica il numero di cifre dedicato alla mantissa (la precisione dell'esponente dipende dall'implementazione)

real e *double precision* hanno precisione prestabilita, una doppia rispetto all'altra.

3.5 Data e Ora

Tipi utilizzati per rappresentare istanti di tempo

```
1 date
2 time [(Precisione)][with time zone]
3 timestamp [(Precisione)][with time zone]
4
```

Ciascun tipo è divisibile in campi:

- *date* ammette come campi *year*, *month* e *day*
- *time* i campi *hour*, *minute*, *second*
- *timestamp* tutti i campi da *year* a *second*

I campi sono divisi da ":".

Ad esempio: 20:03:04+1:00

4 Definizione di Domini

Come nei linguaggi ad alto livello (es. C) è possibile definire nuovi domini (tipi di dati) a partire da quelli predefiniti, anche se il costruttore è più limitato.

```
1 create domain NomeDominio as TipodiDato [ValDefault] [Vincolo]
2
```

Non si possono creare array o strutture poiché il modello relazionale impone che gli attributi siano definiti su un dominio elementare.

È però possibile associare dei vincoli ad un dominio definito dall'utente.

Se si modifica la definizione di un dominio, la modifica si propaga a tutte le tabelle.

4.1 Esempio:

```
1 create domain Voto as smallint
2 default null
3 check ( value >=18 and value <= 30 )
4
```

5 Definizione di Schemi

È possibile definire uno schema di base di dati come collezione di oggetti.

```
1 create [database | schema] [IF NOT EXISTS] db_name [create_option]
2
1 create_option: [DEFAULT] {CHARACTER SET [=] charset_name | COLLATE [=] collation_name |
2 ENCRYPTION [=] {'Y' | 'N'}} }
```

6 Definizione di Tabelle

Una tabella SQL è costituita da una collezione ordinata di attributi e da un insieme di vincoli

```
1 create table NomeTabella
2 (
3     NomeAttributo Dominio [default ValDefault] [Vincoli]
4     {,NomeAttributo Dominio [default ValDefault] [Vincoli]}
5     [AltriVincoli]
6 )
7
```

Una tabella è inizialmente vuota e chi la crea possiede tutti i diritti su di essa.

6.1 Esempio

```
1 create table Dipartimento
2 (
3     Nome char(20) primary key,
4     Indirizzo char(50),
5     Città char(20)
6 )
7
```

7 Valori di default

Si possono specificare valori di default per un attributo quando, al momento della creazione di una nuova tupla, in un campo non viene inserito alcun valore.

```
1 default < GenericoValore | user | null >
2
```

GenericoValore è un valore compatibile col dominio.

user null è il nome dell'utente che aggiorna la tabella corrisponde al valore di default di base.

Un nuovo default (es. nella definizione di un tipo a partire da un altro) sostituisce il precedente.

Nota: è per questo che, anche se, in mancanza di una istruzione di *set default*, a qualsiasi attributo non istanziato viene assegnato come default il valore *null*, è ugualmente previsto il comando *set default null* (di fatto, quindi, per resettare a *null* un precedente diverso default). Es.

```
1 NumeroFigli smallint default 0
2
```

Impone il valore 0 quando non viene specificato il valore dell'attributo.

8 Vincoli intrarelazionali

I più semplici vincoli intrarelazionali predefiniti sono:

- *not null*: indica che il valore nullo non è ammesso su uno specifico attributo.

Si impone che sia inserito in ogni caso un valore, salvo che non sia già definito un valore di default.

SQL non distingue i diversi tipi di valore nullo.

Se è necessario, bisogna definire opportuni domini.

- *unique(Attributo{, Attributo})* indica che l'insieme di attributi deve essere una superchiave per la tabella.
- *primarykey(Attributo{, Attributo})* definisce la chiave primaria.
Può (e deve) essere definita una sola chiave primaria per ogni tabella.
Tutti gli attributi della chiave primaria sono automaticamente *not null*.

8.1 Esempio

```

1  Nome character(20) not null,
2  Cognome character(20) not null,
3  unique (Nome, Cognome)
4

```

Impone che non esistano due tuple in cui sia i valori di *Nome* che di *Cognome* siano uguali

```

1  Nome character(20) not null unique,
2  Cognome character(20) not null unique
3

```

Impone che non esistano due tuple che abbiano uguali valori di *Nome* o di *Cognome*.

8.2 Vincoli di Integrità Referenziale

Il vincolo di **foreign key** (**chiave esterna**) collega i valori di un insieme di attributi della tabella corrente (tabella interna) a un insieme di attributi di un'altra tabella (tabella esterna).

Per ogni tupla della tabella interna i valori della chiave esterna devono essere presenti nei corrispondenti attributi della tabella esterna.

L'insieme di attributi "esterni" deve essere *unique*.

Se la chiave esterna consiste in un unico attributo si usa la clausola *references(AttribTabExt)* nella riga in cui è definito (Mysql); altrimenti *foreign key(ListaAttributi) references(ListaAttributiTabExt)* (sempre applicabile).

8.3 Esempio MySQL

```

1  create table Impiegato
2  (
3      Matricola character(6) primary key,
4      Nome character(20) not null,
5      Cognome character(20) not null,
6      Dipart character(15) references Dipartimento(NomeDip),
7      Stipendio numeric(9) default 0,
8
9      unique (Cognome, Nome),
10     foreign key(Nome, Cognome) references Anagrafica(Nome, Cognome)
11 )
12

```

8.4 Esempio MariaDB

```

1  create table Impiegato
2  (
3      Matricola character(6),
4      Nome character(20) not null,
5      Cognome character(20) not null,
6      Dipart character(15),
7      Stipendio numeric(9) default 0,
8
9      primary key (Matricola),
10     unique (Cognome, Nome),
11     foreign key(Dipart) references Dipartimento(NomeDip),
12     foreign key(Nome, Cognome) references Anagrafica(Nome, Cognome)
13 )
14

```

9 Modifica degli schemi

E' possibile modificare gli schemi con i comandi *alter* e *drop*.

9.1 Comando alter

alter consente di modificare domini e schemi di tabelle.

```
1 alter domain NomeDominio
2 <
3   set default ValDefault |
4   drop default |
5   add constraint DefVincolo |
6   drop constraint NomeVincolo
7 >
```

```
1 alter table NomeTabella
2 <
3   alter column NomeAttributo <set default NuovoDefault | drop default> |
4   add constraint DefVincolo |
5   drop constraint NomeVincolo |
6   add column NomeAttributo |
7   drop column NomeAttributo
8 >
```

NB Quando si inserisce un nuovo vincolo, questo deve essere soddisfatto dai dati già presenti

9.2 Comando drop

Il comando *drop* permette di rimuovere dei componenti

```
1 drop < schema | domain | table | view | assertion > NomeElemento [ restrict | cascade ]
2
```

restrict specifica di non eseguire il comando in presenza di oggetti non vuoti.

cascade implica che gli oggetti specificati siano rimossi insieme a tutti gli oggetti da essi dipendenti.
Quindi, ATTENZIONE!!!!

10 Inserimento

```
1 insert into Dipartimento(NomeDip, Citta) values('Produzione','Torino')
2
```

Si utilizza in genere mediante una form per consentire agli utenti di inserire dati.

L'ordinamento degli attributi (se presente) e dei valori è significativo e le due liste devono avere lo stesso numero di elementi.

```
1 insert into ProdottiMilanesi
2 (
3   select codice, descrizione from Prodotto where LuogoProd = 'Milano'
4 )
5
```

Se in un inserimento non vengono inseriti tutti i dati di una riga si usa o il default (se esiste) o il valore nullo. La corrispondenza fra valori inseriti e attributi è per posizione.

10.1 Esempio

```
1 INSERT INTO Persone VALUES ('Mario',25,52)
2
3 INSERT INTO Persone(Nome, Eta, Reddito) VALUES('Pino',25,52)
4
5 INSERT INTO Persone(Nome, Reddito) VALUES('Lino',55),('Gino',43)
6
7 INSERT INTO Persone ( Nome )
8   SELECT Padre FROM Paternita WHERE Padre NOT IN (SELECT Nome FROM Persone)
9
```

11 Cancellazione

```
1 delete from NomeTabella [ where Condizione ]
2
```

Se *where* non è specificato tutte le righe della tabella vengono eliminate.

Se esiste un vincolo di integrità referenziale con politica di *cascade*... ATTENZIONE!!!!

Siccome *delete* ha la stessa sintassi di *select*, è possibile utilizzare interrogazioni nidificate al suo interno.

```
1 delete from Dipartimento where Nome not in (select Dipart from Impiegato)
2
3 -- NB:
4 delete from Dipartimento -- elimina il contenuto della tabella
5 drop table Dipartimento cascade -- altera lo schema
6
```

12 Modifica

```
1 update NomeTabella
2 set Attributo = <Espressione|SelectSQL|null|default> {, Attributo = <Espressione|SelectSQL
3 |null|default>} [ where Condizione ]
```

Aggiorna uno o più attributi delle righe di *NomeTabella* che soddisfano l'eventuale *Condizione*.

Se non c'è condizione la modifica avviene per tutte le righe.

Il nuovo valore può essere:

- Il risultato di un'espressione valutata sugli attributi della tabella
- Il risultato di una generica interrogazione SQL
- Il valore nullo
- Il valore di default

```
1 update Dipendente
2 set Stipendio = StipendioBase + 5 where Matricola = 'XYZ' -- aggiorna una sola riga (
3 Matricola e' chiave)
4
5 update Dipendente
6 set Stipendio = Stipendio * 1.1 where Dipart = 'Amministrazione' -- aggiorna un insieme di
7 righe
```

Nella valutazione delle espressioni bisogna ricordarsi che SQL ha una natura orientata agli insiemi e non alle tuple.

Quindi non è possibile pensare all'esecuzione di un comando come esecuzione riga per riga, ma come un'operazione effettuata contemporaneamente su tutto un insieme.

12.1 Esempio

Vogliamo aumentare del 10% gli stipendi degli impiegati con stipendio inferiore ai 30 e del 15% quello degli impiegati con stipendio superiore ai 30.

Attenzione! Se scriviamo

```
1 update Impiegato set Stipendio = Stipendio*1.1 where Stipendio <= 30
2
3 update Impiegato set Stipendio = Stipendio*1.15 where Stipendio > 30
4
```

Ad alcuni impiegati lo stipendio viene aumentato 2 volte!!!

Se invertiamo l'ordine la procedura risulta corretta.

Ma non sempre è così facile.

13 Interrogazioni

L'istruzione base per le interrogazioni è *select*.

```
1 select ListaAttributi (target list) from ListaTabelle (clausola from) [ where Condizione ]
2 (clausola where)
```

Più in dettaglio:

```
1 select AttrEspr [[as]Alias]{,AttrEspr [[as]Alias]} from Tabella [[as]Alias]{,Tabella [[as]
2 Alias]} [ where Condizione ]
```

- Fa il prodotto cartesiano fra le tabelle in *ListaTabelle*
- Seleziona le righe che soddisfano *Condizione*
- Proietta il risultato sugli attributi specificati da *ListaAttributi*

Ogni colonna (tabella) può essere ridenominata con un alias.

13.1 Esempio

Data una base di dati che contiene le tabelle:

IMPIEGATO (Nome, Cognome, Dipart, Ufficio, Stipendio, Città)

DIPARTIMENTO (Nome, Indirizzo,Città)

```
1 select Stipendio/12 as SalarioMensile from Impiegato where Cognome = 'Rossi'
2
```

Genera una tabella con una colonna ridenominata *SalarioMensile* e tante righe quanti sono gli impiegati che si chiamano Rossi.

Se si indica *select** si selezionano tutti gli attributi.

13.2 Interpretazione algebrica delle interrogazioni

```
1 select T_1.Attributo_11, ... T_h.Attributo_hm from Tabella_1 as T_1, ... Tabella_h as T_h
2 where Condizione
```

equivale a

$$\Pi_{T_1.Attributo_{11} \dots T_h.Attributo_{hm}}(\sigma_{Condizione}(Tabella_1 \times Tabella_2 \times \dots \times Tabella_h))$$

$\times$ = prodotto cartesiano

Persone		
Nome	Età	Reddito
Andrea	27	21
Aldo	25	15
Maria	55	42
Anna	50	35
Filippo	26	30
Luigi	50	40
Franco	60	20
Olga	30	41
Sergio	85	35
Luisa	75	87

Vogliamo il Nome e il Reddito delle persone che hanno meno di 30 anni:

$$PROJ_{Nome, Reddito}(SEL_{Eta < 30}(Persone))$$

Che in SQL è:

```
1 select nome, reddito from persone where eta < 30
2
3 select p.nome as nome, p.reddito as reddito from persone as p where p.eta < 30
4
```

13.3 Interrogazioni su più tabelle

Se si vogliono estrarre informazioni da più tabelle, queste devono apparire come argomento della clausola *from* o in forma di lista o esplicitamente come (theta-)join fra le tabelle stesse.

Infatti, la clausola *from* denota la singola tabella a cui applicare le operazioni di proiezione (specificate dalla clausola *select*) e selezione (specificate dalla clausola *where*) per ottenere il risultato voluto.

Se *from* viene applicata a una lista di tabelle, denota il prodotto cartesiano fra tali tabelle.

In pratica tale situazione indica che si possono combinare tuple provenienti da qualsiasi tabella, senza che debbano soddisfare alcun vincolo interrelazionale.

Se si deve formulare un theta-join fra le tabelle, si può fare in modo esplicito: *from* reggerà allora un *join* fra le tabelle (*join* **esplicito**: A *join* B *on* Condizione).

```
1 select Studenti.Nome, Studenti.Cognome, Esami.voto from Studenti join Esami on Esami.
2 studente=Studenti.matricola
```

Un theta-join equivale a un prodotto cartesiano seguito da una selezione (sulla condizione di *join*): su questa base un *join* **implicito** specifica solo la lista delle tabelle nella clausola *from* (prodotto cartesiano) e inserisce la Condizione di *join* nella clausola *where* (selezione).

```
1 select Studenti.Nome, Studenti.Cognome, Esami.voto from Studenti, Esami where Esami.
2 studente=Studenti.matricola
```

Estrarre i nomi degli impiegati e le città dove lavorano:

```
1 select Impiegato.Nome, Impiegato.Cognome, Dipartimento.Citta from Impiegato, Dipartimento
2 where Impiegato.Dipart=Dipartimento.Nome
```

oppure (con join esplicito):

```
1 select Impiegato.Nome, Impiegato.Cognome, Dipartimento.Citta from Impiegato join
2 Dipartimento on Impiegato.Dipart=Dipartimento.Nome
```

È possibile omettere il nome della tabella per quegli attributi che non presentano ambiguità.

```
1 select Impiegato.Nome, Cognome, Dipartimento.Citta from Impiegato, Dipartimento where
2 Dipart = Dipartimento.Nome
```

e abbreviare ulteriormente il codice utilizzando gli alias:

```
1 select I.Nome, Cognome, D.Citta from Impiegato as I, Dipartimento as D where Dipart = D.
2 Nome
```

14 Clausola where

Ha come argomento una condizione logica, che ammette come operatori i predicati semplici (confronto attributo-costante o attributo-espressione): =, <>, <, >, <=, >=

I predicati semplici possono essere modificati tramite gli operatori logici *and*, *or*, *not*.

not ha precedenza su *and* e *or*.

La precedenza fra *and* e *or* non è definita: quando si coordinano più predicati con *and* e *or* è bene esplicitare sempre le precedenze con le parentesi.

```
1 select Nome from Impiegato where Cognome = 'Rossi' and (Dipart='Amministrativ' or Dipart='
2 Produz')
```

15 Operatore like

Per i confronti fra stringhe è definito anche l'operatore *like*.

Il confronto è effettuato con una stringa che può contenere i caratteri speciali % e _.

_ rappresenta un carattere arbitrario.

% rappresenta in numero arbitrario di caratteri (anche zero).

```
1 select * from Impiegato where Cognome like '_o%i' -- La condizione e' soddisfatta da Rossi
2 Borronei Poli Pollastri ecc.
```

16 Gestione dei valori nulli

La gestione dei valori nulli, a seconda dell'implementazione, avviene attraverso una logica a due valori come in SQL-89, o a tre valori (vero, falso, unknown) come in SQL-2.

In MySQL logica a due valori con gestione del valore nullo.

Le condizioni sui valori nulli possono essere definite attraverso i predicati *is null* e *is not null*.

Paternità		Maternità	
Padre	Figlio	Madre	Figlio
Sergio	Franco	Luisa	Maria
Luigi	Olga	Luisa	Luigi
Luigi	Filippo	Anna	Olga
Franco	Andrea	Anna	Filippo
Franco	Aldo	Maria	Andrea
		Maria	Aldo

NB Si suppone che su Figlio sia applicato il vincolo *unique*

17 Join Implicito ed Esplicito

Basandoci sulle tabelle Paternità e Maternità prendiamo Padre e Madre di ogni persona.

Con Join Implicito (Prodotto Cartesiano + selezione):

```
1 select paternita.figlio, padre, madre
2 from maternita, paternita
3 where paternita.figlio = maternita.figlio
4
```

Con Join Esplicito (Theta-Join):

```
1 select paternita.figlio, padre, madre
2 from maternita join paternita on paternita.figlio = maternita.figlio
3
```

18 Duplicati

Al contrario di quanto avviene per l'algebra relazionale SQL ammette duplicati.

Quindi nella lista di città generata da

```
1 select Citta
2 from Persona
3 where Cognome = 'Rossi'
4
```

Il nome di una città può comparire più volte.

Per evitare i duplicati SQL prevede la clausola *distinct* da inserire prima dell'attributo che non si vuole duplicare

```
1 select distinct Citta
2 from Persona
3 where Cognome = 'Rossi'
4
```

19 Join: sintassi

In SQL-2 è stata introdotta la seguente sintassi per esprimere il join ed estenderlo ai join esterni:

```
1  select AttrEspr [[as]Alias]{,AttrEspr [[as]Alias]}
2  from Tabella [[as]Alias] {[TipoJoin]joinTabella [[as]Alias] on CondizioneJoin}
3  [ where AltraCondizione ]
4
```

TipoJoin può assumere i valori:

- inner
- right[outer]
- left[outer]
- full[outer]

inner è il **default** e corrisponde a un theta-join.

Il tipo natural (join naturale) implica la condizione di uguaglianza degli attributi con lo stesso nome.

20 Alias e Variabili

L'uso degli alias consente di:

- Compattare il codice
- Fare riferimento a più esemplari della stessa tabella
- Creare interrogazioni nidificate con riferimenti fra la query esterna e quella interna

Se l'interrogazione coinvolge una singola istanza di una tabella non c'è differenza fra variabile ed alias.

Se utilizza più istanze si parla più propriamente di variabile.

20.1 Esempio di alias usato come variabile

Impiegati il cui nome è anche il cognome di un altro impiegato del dipartimento Produzione.

```
1  select I1.Cognome, I1.Nome
2  from Impiegato I1, Impiegato I2
3  where I1.Nome = I2.Cognome
4  and I1.Nome <> I2.Nome
5  and I2.Dipart = 'Produzione'
6
```

21 Ordinamento

È possibile ordinare le righe del risultato di una interrogazione attraverso la clausola *order by*, a chiusura dell'interrogazione.

```
1  order by AttrdiOrdinamento [asc | desc] {, AttrdiOrdinamento [asc | desc]}
2
3  asc -- (default) indica ordinamento ascendente,
4  desc -- discendente.
5
```

NB l'ordinamento alfabetico è ascendente ($a < b < c < \dots < y < z$)

Il primo attributo ha priorità, a parità si ordina sulla base del secondo, ecc.

```
1  select *
2  from Persona
3  order by Cognome, Nome
4
```

22 Operatori Aggregati

In algebra relazionale le espressioni vengono valutate sulle singole tuple in successione.

Talvolta però possono essere necessarie informazioni derivabili dall'esame di tutte le tuple o di sottoinsiemi di tuple.

SQL prevede una serie di operatori aggregati:

- count
- sum
- max
- min
- avg

Sintassi:

```
1 count (< * | [distinct|all] ListaAttributi >) <sum|max|min|avg>([distinct|all] AttrEspr)
2
```

Determinare il numero degli impiegati che si chiamano Rossi

```
1 select count(*)
2 from Impiegato
3 where nome= 'Rossi'
4
```

```
1 select count(arg) from ... -- esegue l'interrogazione come se fosse select arg from ... e
   restituisce il numero di tuple di cui e' composto il risultato
2
3 select sum(arg), max(arg), min(arg), avg(arg) -- restituiscono la somma, il massimo, il
   minimo, la media del risultato di select arg from ...
4
```

23 Interrogazioni con raggruppamento

Gli operatori aggregati vengono calcolati sulla base di **tutte** le righe che vengono prodotte come risultato dell'interrogazione.

SQL non ammette che nella stessa *target list* compaiano funzioni aggregate ed espressioni a livello di riga, come il nome di un attributo.

Non è infatti possibile associare un singolo risultato, calcolato su tutte le righe, a un attributo che, se ad es. è chiave, può anche assumere valori tutti diversi in ogni riga

SQL non ammette che nella stessa *target list* compaiano funzioni aggregate e nomi di attributi, poiché l'operatore di aggregazione agisce su tutte le righe della tabella, mentre un attributo è valutato singolarmente per ogni tupla.

```
1 select nome, max(reddito)
2 from persone
3
```

Di chi è il nome? *select nome* dovrebbe restituire tutti i nomi, *max()* un singolo valore.

La target list deve essere **omogenea**.

L'interrogazione è corretta solo se la *target list* contiene solo operatori di aggregazione, anche più di uno: Es.

```
1 select min(eta), avg(reddito)
2 from persone
3 where reddito > 100
4
```

Restituisce due valori singoli, entrambi calcolati su tutte le tuple della tabella persone: l'età minima e il reddito medio di tutte le persone con reddito > 100.

23.1 group by

In alcuni casi è utile/necessario applicare l'operatore separatamente ad uno o più sottoinsiemi delle righe. L'operatore *group by Attr*, *Attr* specifica come suddividere le tabelle in sottoinsiemi omogenei per valore, su cui valutare separatamente l'operatore aggregato.

```
1 select Dipart, sum(Stipendio)
2 from Impiegato
3 group by Dipart
4
```

(per ogni valore x assunto da *Dipart* calcola la somma dei valori di *Stipendio* di tutte le tuple con $Dipart = x$)

Se è presente un raggruppamento, nella target list possono comparire anche attributi, ma devono costituire un sottoinsieme di quelli usati nella clausola *group by*

Es. (scorretto)

```
1 select Ufficio
2 from Impiegato
3 group by Dipart
4
```

Poiché deve venire prodotta una sola riga per ogni valore di *Dipart*, e, a parità di tale valore, possono aversi diversi valori di *Ufficio*, il risultato dell'interrogazione è indeterminato.

Se per l'attributo che è nella target list senza comparire nella clausola *group by* esiste una dipendenza funzionale dagli attributi su cui si raggruppa (cioè se a ciascun valore di questi ultimi corrisponde un solo valore per tale attributo) la query potrebbe fornire un risultato corretto, ma per semplicità SQL (n.b. ma non MySQL) lo vieta comunque.

24 Operatori aggregati e Raggruppamenti

Il numero di figli di ciascun padre:

```
1 select Padre, count(*) as NumFigli
2 from paternita
3 group by Padre
4
```

Paternità			
Padre	Figlio	Padre	NumFigli
Sergio	Franco	Sergio	1
Luigi	Olga	Luigi	2
Luigi	Filippo	Franco	2
Franco	Andrea		
Franco	Aldo		

25 Raggruppamenti e Target List

Interrogazione **scorretta**:

```
1 select padre, avg(f.reddito), p.reddito
2 from persone f join paternita on figlio = f.nome join persone p on padre = p.nome
3 group by padre
4
```

In effetti, un vincolo di integrità referenziale fra *padre* e *persone.nome* può fare sì che il valore di *persone.reddito* da mostrare si riferisca davvero alla corretta istanza di *padre*, attributo su cui si raggruppa. Però viene comunque considerato errore.

Interrogazione **corretta**

```
1 select padre, avg(f.reddito), p.reddito
2 from persone f join paternita on figlio = nome join persone p on padre = p.nome
3 group by padre, p.reddito
4
```

26 Condizioni sui gruppi

Può essere anche necessario effettuare una selezione sui risultati di una query con raggruppamento.

Se le condizioni sono verificabili a livello di singole righe e non coinvolgono i risultati del raggruppamento basta utilizzare la clausola *where*, altrimenti si estende la clausola *group by* con l'estensione *having*, che applica la condizione di selezione al risultato di ogni gruppo.

```
1  select Dipart, sum(Stipendio) as SommaStipendi
2  from Impiegati
3  group by Dipart
4  having SommaStipendi > 100
5
```

Se non si specifica *group by* e si usa *having* da solo la condizione è applicata all'insieme di tutte le righe. Se la condizione non è verificata, il risultato è vuoto.

26.1 Esempio

I padri i cui figli hanno un reddito medio maggiore di 25

```
1  select padre, avg(f.reddito)
2  from persone f join paternita on figlio=nome
3  group by padre having avg(f.reddito) > 25
4
```

26.2 Where o Having?

I padri i cui figli sotto i 30 anni hanno un reddito medio maggiore di 25

```
1  select padre, avg(f.reddito)
2  from persone f join paternita on figlio = nome where eta < 30
3  group by padre having avg(f.reddito) > 25
4
```

27 Select

La sintassi di *select* a cui siamo arrivati dopo le estensioni viste è quindi:

```
1  SelectSQL ::=
2  select ListaAttributiOEspressioni
3  from ListaTabelle
4  [ where CondizioniSemplici ]
5  [ group by ListaAttributiDiRaggruppamento ]
6  [ having CondizioniAggregate ]
7  [ order by ListaAttributiDiOrdinamento ]
8
```

28 Interrogazioni di tipo insiemistico

SQL fornisce anche gli operatori di tipo insiemistico

- union
- intersect
- except
- minus (ma non in MySQL)

con la seguente sintassi:

```
1  SelectSQL
2  {< union | intersect | except > [all] SelectSQL}
3
```

- *intersect* e *except* potrebbero anche essere derivati attraverso opportune query
- Gli operatori insiemistici eseguono per default una eliminazione dei duplicati; *all* specifica di non farla
- Gli schemi su cui si opera non devono essere identici come nell'algebra relazionale ma avere uguale numero di attributi e domini compatibili.
La corrispondenza è per posizione.

28.1 Esempi

Estrarre nomi e cognomi degli impiegati

```
1 select Nome
2 from Impiegato
3 union
4 select Cognome
5 from Impiegato
6
```

Estrarre i cognomi degli impiegati che sono anche nomi

```
1 select Nome
2 from Impiegato
3 intersect
4 select Cognome
5 from Impiegato
6
```

29 Interrogazioni nidificate

È possibile anche realizzare clausole *where* in cui il confronto non avviene fra predicati semplici o fra valori, ma fra valori e insiemi di valori, appartenenti allo stesso dominio, che sono risultato di un'altra interrogazione che viene annidata all'interno di quella "principale".

Di qui deriva il termine query nidificate.

SQL offre 2 possibilità di estendere gli operatori di confronto al caso del confronto valore/insieme (risultato query):

- *all* specifica che il risultato del confronto è vero se è vero per tutte le righe del risultato dell'interrogazione nidificata.
- *any* (default) specifica che il risultato del confronto è vero se è vero per una qualunque riga del risultato dell'interrogazione.

```
1 select I.Nome, Cognome
2 from Impiegato I
3 where Dipart = any (select D.Nome
4 from Dipartimento D
5 where Citta='Firenze')
6
```

Il risultato comprende tutte le righe di *Impiegato* per cui il valore di *Dipart* è uguale ad almeno uno dei valori di *Nome* in *Dipartimento*, limitatamente alle tuple di *Dipartimento* per cui *Citta* = 'Firenze', cioè l'elenco degli impiegati che lavorano in dipartimenti con sede a Firenze.

Lo stesso risultato si poteva ottenere con un *join*, ma così, specialmente per interrogazioni complesse, è più leggibile.

```
1 select I.Nome, Cognome
2 from Impiegato I
3 where Dipart = any (select D.Nome
4 from Dipartimento D
5 where Citta = 'Firenze')
6 select I.Nome, Cognome
7 from Impiegato I, Dipartimento D
8 where Dipart = D.Nome and Citta = 'Firenze'
9
```

Nome e reddito del padre di Franco

```
1 select Nome, Reddito
2 from Persone, Paternita
3 where Nome = Padre and Figlio = 'Franco'
4 select Nome, Reddito
5 from Persone
6 where Nome =(select Padre
7 from Paternita
8 where Figlio = 'Franco')
9
```

29.1 Interrogazioni nidificate per gli alias

L'uso delle interrogazioni nidificate può anche eliminare la necessità degli alias.
Es.

```
1 select I1.Nome
2 from Impiegato I1, Impiegato I2
3 where I1.Nome = I2.Nome and I1.Dipart = 'Produz' and I2.Dipart <> 'Produz'
```

equivale a

```
1 select Nome
2 from Impiegato I1
3 where Dipart = 'Produz' and Nome = any (select I2.Nome
4 from Impiegato I2
5 where I2.Dipart <> 'Produz')
```

29.2 Interrogazioni nidificate e join

Non tutte le interrogazioni nidificate corrispondono ad un join

```
1 select Nome
2 from Dipartimento
3 where Nome <> all (select Dipart
4 from Impiegato
5 where Cognome='Rossi')
```

La condizione è verificata per le righe che NON contengono un certo valore, quindi non è esprimibile mediante un join, che richiede una corrispondenza fra valori.
Però è equivalente a :

$$\Pi_{Nome}(DIPARTIMENTO) - \Pi_{Dipart}(\sigma_{Cognome='Rossi'}(IMPIEGATO))$$

e quindi le due query potevano essere unite da *except*

NB Le condizioni "*= any*" e "<> *all*" si possono anche indicare come "*in*" e "*not in*".

29.3 Operatori Aggregati e Nidificazione

La persona (o le persone) con il reddito massimo

```
1 select *
2 from persone
3 where reddito = ( select max(reddito)
4 from persone )
```

IMPORTANTE!!! *max* non è una funzione ma un operatore SQL di aggregazione, quindi scrivere *whereredito = max(reddito)* sarebbe un errore perché gli operatori di aggregazione lavorano su insiemi di tuple.

Una funzione è invece valutata a livello di tupla.

*where reddito <= 10000 * sin(n_civico)* è accettabile perché confronta i valori di *n_civico* e l'espressione basata su *reddito* per ogni singola tupla di *persone*.

```
1 select *
2 from persone
3 where reddito < ( select reddito
4 from persone )
```

Confronta il valore di reddito di ogni tupla della tabella persona con TUTTI i valori di reddito presenti nella tabella persone.

Siccome *any* è il default, estrae tutti i valori di reddito ad esclusione del massimo.

```
1 select *
2 from persone
3 where reddito < reddito
```

Restituisce un insieme vuoto perché il valore di un attributo non può essere minore di se stesso (si valuta una tupla alla volta).

```

1  select Dipart, Stipendio
2  from Impiegato
3  where Stipendio = ( select max(Stipendio)
4                    from Impiegato )
5

```

equivale a

```

1  select Dipart, Stipendio
2  from Impiegato
3  where Stipendio >= all ( select Stipendio
4                        from Impiegato )
5

```

(N.B. il massimo di un insieme è il valore, appartenente a quell'insieme, che è maggiore o uguale a tutti gli altri)

La somma degli stipendi degli impiegati di ciascun Dipartimento

```

1  select Dipart, sum(Stipendio)
2  from Impiegato
3  group by Dipart
4

```

Il Dipartimento per cui la somma degli stipendi è più alta

```

1  select Dipart, sum(Stipendio) as Costodip
2  from Impiegato
3  group by Dipart
4  ??? CostoDip >= all (select ???)
5

```

La somma degli stipendi degli impiegati di ciascun Dipartimento

```

1  select Dipart, sum(Stipendio)
2  from Impiegato
3  group by Dipart
4

```

Il Dipartimento per cui la somma degli stipendi è più alta

```

1  select Dipart, sum(Stipendio) as Costodip
2  from Impiegato
3  group by Dipart
4  having CostoDip >= all (select sum(Stipendio)
5                        from Impiegato
6                        group by Dipart)
7

```

29.4 Interpretazione delle query nidificate

Per analizzare il risultato di una interrogazione con nidificazione si può supporre di valutare prima il risultato dell'interrogazione nidificata (query interna), "sostituendo" poi tale valore nella query esterna per ottenere il risultato finale.

Questo procedimento è anche efficiente, in quanto l'interrogazione nidificata viene eseguita una sola volta.

Tuttavia, è possibile farlo solo se le due query sono indipendenti fra loro, cioè se la query interna non varia al variare della specifica tupla della query esterna che si deve confrontare col risultato della query interna.

Quando esiste questa dipendenza, espressa tramite l'uso di un alias comune alle due query, si parla di **passaggio di binding** fra le due interrogazioni.

Se esiste un riferimento (passaggio di binding) fra le due query, bisogna usare l'interpretazione "procedurale" di query, che calcola il prodotto cartesiano fra le tabelle indicate nella clausola *from* e poi verifica la condizione specificata nella clausola *where* separatamente per ogni riga.

Quindi per ogni riga della query esterna (da cui dipende anche la query interna) si valuta prima il risultato della corrispondente query interna, per poi calcolare il predicato logico sulla query esterna.

Il passaggio di binding avviene mediante le variabili SQL, cioè alias condivisi fra più interrogazioni.

Un alias è utilizzabile nella query in cui è definito o nell'ambito di una query nidificata al suo interno.

Quindi la query interna può ereditare (condividere) gli alias di quella esterna.

Due query allo stesso livello non possono condividere variabili.

Di fatto, si realizzano n theta-join separati fra ciascuna delle n righe risultanti dalla query esterna e la tabella risultante dall'esecuzione della query interna, istanziando gli alias all'interno di quest'ultima coi valori corrispondenti della query esterna.

Es.

L'insegnamento cui si riferisce il primo esame sostenuto da ogni studente

```
1  select S.cognome, S.nome, E.insegnamento
2  from Studenti S, Esami E
3  where S.Matricola = E.Studente
4  and E.Data = ( select min(E1.data)
5  from Esami E1
6  where E1.Studente = S.Matricola )
7
```

30 exists

exists è un operatore logico applicabile a query nidificate.

Restituisce vero se la query che regge dà un risultato non nullo, falso se è nullo.

È utilizzabile in modo significativo solo se esiste un passaggio di binding fra interrogazione esterna e interrogazione nidificata.

Es.

Le persone che hanno almeno un figlio

```
1  select *
2  from Persone P
3  where exists ( select *
4  from Paternita
5  where Padre = P.Nome)
6  or exists ( select *
7  from Maternita
8  where Madre = P.Nome)
9
```

```
1  select *
2  from Persona P
3  where exists ( select *
4  from Persona P1
5  where P1.Nome = P.Nome
6  and P1.Cognome = P.Cognome
7  and P1.CFiscale <> P.CFiscale)
8
```

In questi casi non è possibile eseguire prima la query interna, in quanto è indeterminata se non si risolve il riferimento (passaggio di binding).

Quindi per ogni riga dell'interrogazione esterna dovrà essere valutata una diversa interrogazione nidificata.

31 Costruttore di tuple

Quando si deve valutare una condizione che coinvolge un insieme di attributi, la lista degli attributi coinvolti viene inserita all'interno di una parentesi tonda.

Es.

```
1  select *
2  from Persona P
3  where (Nome, Cognome) in
4  (select Nome, Cognome
5  from Persona P1
6  where P.CFiscale <> P1.CFiscale)
7
```

32 Vincoli di integrità generici

Con i costrutti visti sinora, non è sempre possibile definire tutti i possibili vincoli di integrità.

Per questo esiste l'istruzione

check(Condizione)

La condizione specificata deve essere verificata da tutte le tuple in ogni momento.

È possibile con *check* definire tutti i vincoli predefiniti.

Però è meno leggibile e non si possono applicare le politiche di reazione alle violazioni.

Tuttavia è molto potente.

32.1 Esempio check

```
1  create table Impiegato
2  (
3      Matricola character(6),
4      Cognome  character(20),
5      Nome    character(20),
6      Sesso  character not null
7      check (Sesso in ('M','F'))
8      Stipendio integer,
9      Superiore character(6),
10     check( Stipendio <=(select Stipendio
11                        from Impiegato J
12                        where Superiore=J.Matricola) )
13 )
14
```

NB MySQL non ammette query dentro una *check*

33 Asserzioni

Le asserzioni sono vincoli che fanno parte dello schema; non sono associati ad alcun attributo o tabella.

Permettono di definire tutti i vincoli utilizzabili nella definizione di una tabella, ma anche vincoli su più tabelle o vincoli che richiedono che una tabella abbia certe caratteristiche (es. una certa cardinalità).

```
1  create assertionNomeAsserzione check(Condizione)
2
```

Es.

```
1  create assertion AlmenoUnImpiegato
2  check (1 <= (select count(*)
3              from Impiegato))
4
```

NB MySQL ammette solo: *constraint Nome check(Cond)* in una istruzione *create table*.

Ogni vincolo di integrità è associato ad una politica di controllo che specifica se è immediato o differito.

Se è **immediato** è verificato immediatamente dopo una modifica.

Se è **differito**, si verifica solo al termine dell'esecuzione di una serie di operazioni che si considerano parte di un'unica azione atomica (transazione).

Se un vincolo immediato viene violato, l'operazione di modifica può essere annullata immediatamente (**rollback parziale**).

Tutti i vincoli predefiniti sono immediati.

Se invece è differito, al momento in cui si verifica la violazione non è più possibile identificare l'operazione che l'ha causata e quindi va annullata tutta la transazione (**rollback totale**).

Questo garantisce la consistenza della base di dati.

Per cambiare il tipo di controllo:

```

1  set constraint [NomeVincolo] immediate
2  set constraint [NomeVincolo] deferred -- (deferred)
3

```

34 Viste

Le viste sono tabelle virtuali ricavate da informazioni contenute in altre tabelle.

Nella definizione possono essere contenute anche altre viste purché non vi siano dipendenze ricorsive o immediate (una vista non può dipendere da se stessa), né transitive.

```

1  create view NomeVista[(ListaAttributi)] as
2  SelectSQL
3  [ with [ local | cascaded ] check option ]
4

```

- La query SQL deve restituire un numero di attributi pari a quelli contenuti nello schema
- L'ordine degli attributi nella target list deve rispettare quello dello schema

```

1  create view
2  ImpiegatiAmmin(Matricola, Nome, Cognome, Stipendio)
3  as
4  select Matricola, Nome, Cognome, Stipendio
5  from Impiegato
6  where Dipart = 'Amministrazione'
7  and Stipendio > 10
8

```

Su certe viste è possibile fare modifiche che alterano le tabelle che le compongono.

Ci sono problemi se la vista è definita tramite un join.

SQL permette la modifica di una vista solo se una sola riga di ciascuna tabella di base corrisponde a una riga della vista.

Di solito si richiede che sia definita su una sola tabella e/o che contenga almeno una chiave primaria.

check option consente di fare modifiche sulle righe della vista solo se le righe continuano ad appartenere alla vista dopo le modifiche.

Se una vista è definita in termini di altre viste l'opzione **local** o **cascaded** specifica se il controllo debba coinvolgere solo la vista più esterna o se deve essere propagato a tutti i livelli.

Il default è *cascaded*.

Quindi, se è specificata la *check option*, ogni comando di aggiornamento per essere propagato non deve eliminare righe dalla vista.

Le viste consentono anche di sostituire interrogazioni nidificate o di creare interrogazioni altrimenti impossibili da definire.

Es.

```

1  create view BudgetStipendi(Dip, TotStipendi)
2  as
3  select Dipart, sum(Stipendio)
4  from Impiegato
5  group by Dipart
6

```

Trovare il dipartimento che spende di più in stipendi

```

1  select Dip
2  from BudgetStipendi
3  where TotStipendi=(select max(TotStipendi)
4  from BudgetStipendi)
5

```

35 Controllo degli accessi

SQL prevede la definizione di utenti, a ciascuno dei quali sono assegnati privilegi diversi.

Gli utenti possono essere gli stessi del sistema su cui è attivo il server SQL, oppure indipendenti dal sistema.

Ogni componente del sistema è proteggibile, di solito si proteggono le tabelle.

Il controllo degli accessi è basato sul concetto di privilegio, caratterizzato da:

- risorsa cui si riferisce
- utente che concede il privilegio
- utente che lo riceve
- azione che viene permessa
- possibilità di trasmettere o meno il privilegio ad altri utenti

Il creatore di una risorsa ha tutti i privilegi, al momento della sua creazione.

Esiste anche un **utente “speciale”** *_system* (*root* in MySQL), che ha tutti i privilegi su tutte le risorse in qualsiasi momento.

I principali privilegi sono:

- *insert* (applicabile a tabelle o viste) inserisce un nuovo oggetto nella risorsa
- *update* (tabelle viste attributi) aggiorna il valore di un oggetto
- *delete* (tabelle e viste) rimuove un oggetto
- *select* (tabelle viste attributi) permette di leggere la risorsa
- *references* (tabelle ed attributi) permette che venga fatto riferimento ad una risorsa nell'ambito della definizione dello schema di una tabella
- *usage* (domini) permette che venga usata la risorsa

drop e *alter* sono **riservati al creatore** degli oggetti cui si applicano.

I comandi per concedere o revocare **privilegi** sono *grant* e *revoke*.

```
1 grant Privilegi on Risorsa to Utenti [with grant option]
2
```

Questo codice concede i Privilegi sulla Risorsa agli Utenti.

```
1 grant select on Dipartimento to Stefano
2
```

Questo codice concede all'utente Stefano il privilegio di select sulla tabella Dipartimento.

Se si specifica anche *with grant option* l'utente può propagare i diritti anche ad altri.

La parola chiave *all privileges* specifica tutti i possibili privilegi.

```
1 revoke Privilegi on Risorsa from Utenti [restrict|cascade]
2
```

Questo codice fa l'operazione inversa.

Con *restrict* si impedisce che la revoca provochi altre revoke, con *cascade* si concede.